25 AÑOS

PUCP

Ingeniería de las Telecomunicaciones

Spring Framework 5 & Sprint Boot 2

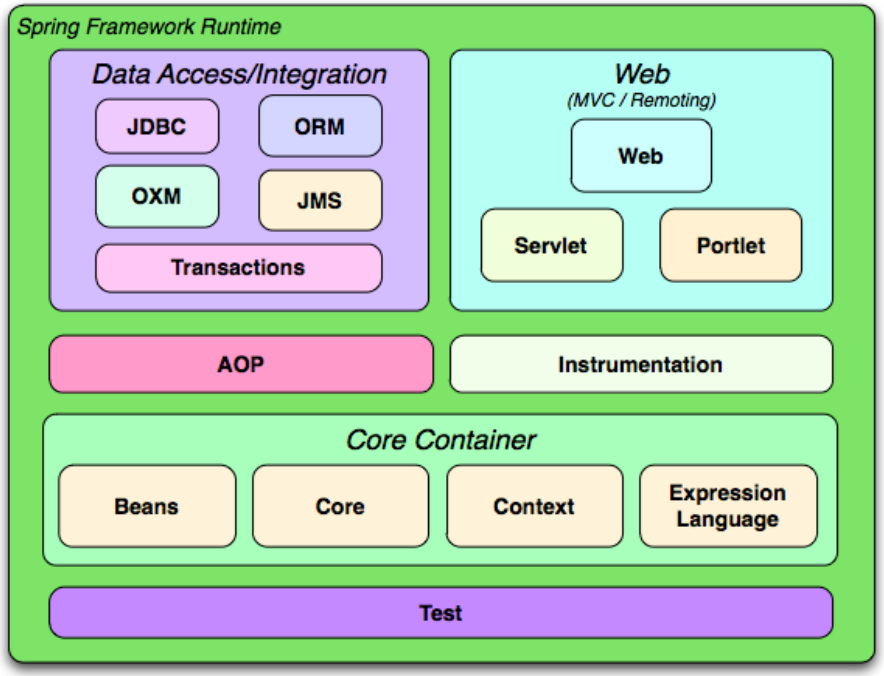
25 AÑOS
Ingeniería de las Telecomunicaciones

Definición

- Spring Framework es una plataforma para desarrollar aplicaciones empresariales en Java.
- Este framework gestiona toda la infraestructura de una aplicación, permitiendo al programador centrarse en las funcionalidades y la lógica de la misma.
- Es el “pegamento” que une todos los componentes, gestiona su ciclo de vida y la interacción entre ellos.

Modelo de spring

- Spring contiene un gran número de módulos, mayor información en:
- https://docs.spring.io/spring-framework/reference/overview.html



Spring MVC

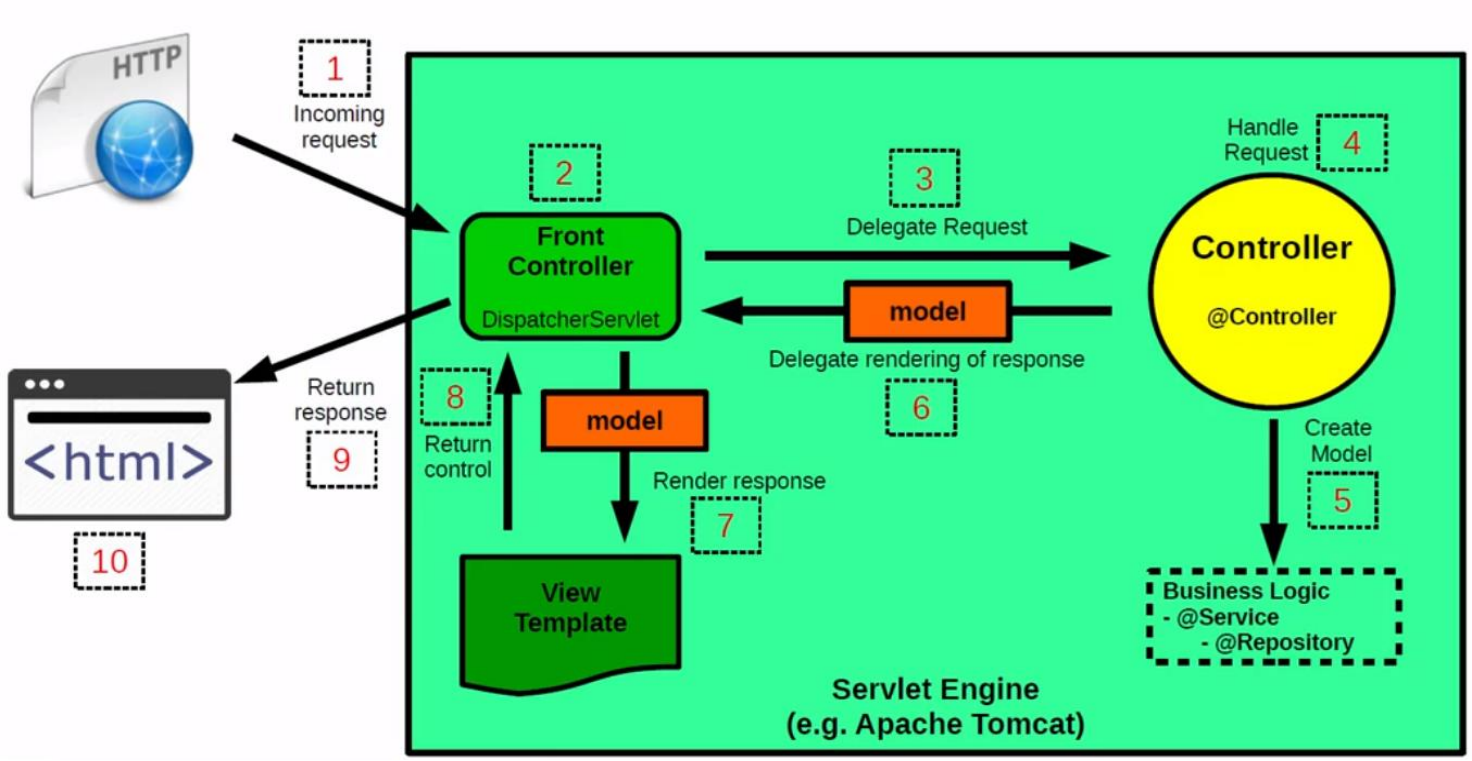
Spring MVC también conocido como **Spring Web MVC** es un framework web basado en Servlets que viene incluido en Spring Framework (spring-webmvc).
Será el módulo principal del framework Spring que se utilizará para desarrollar aplicaciones web.

Permite la separación en tres capas:

- **Modelo**: **Repositorios, Entidades, JPA** (SW1 → Daos, Beans, JDBC)
- **Vista**: **thymeleaf** (SW1 → jsp)
- **Controlador**: (**@Controllers**)

Arquitectura Spring MVC

- Spring MVC está diseñado siguiendo el patrón de diseño de **Front Controller.**
- En Spring MVC, el **Front Controller** es mejor conocido como **DispatcherServlet.**



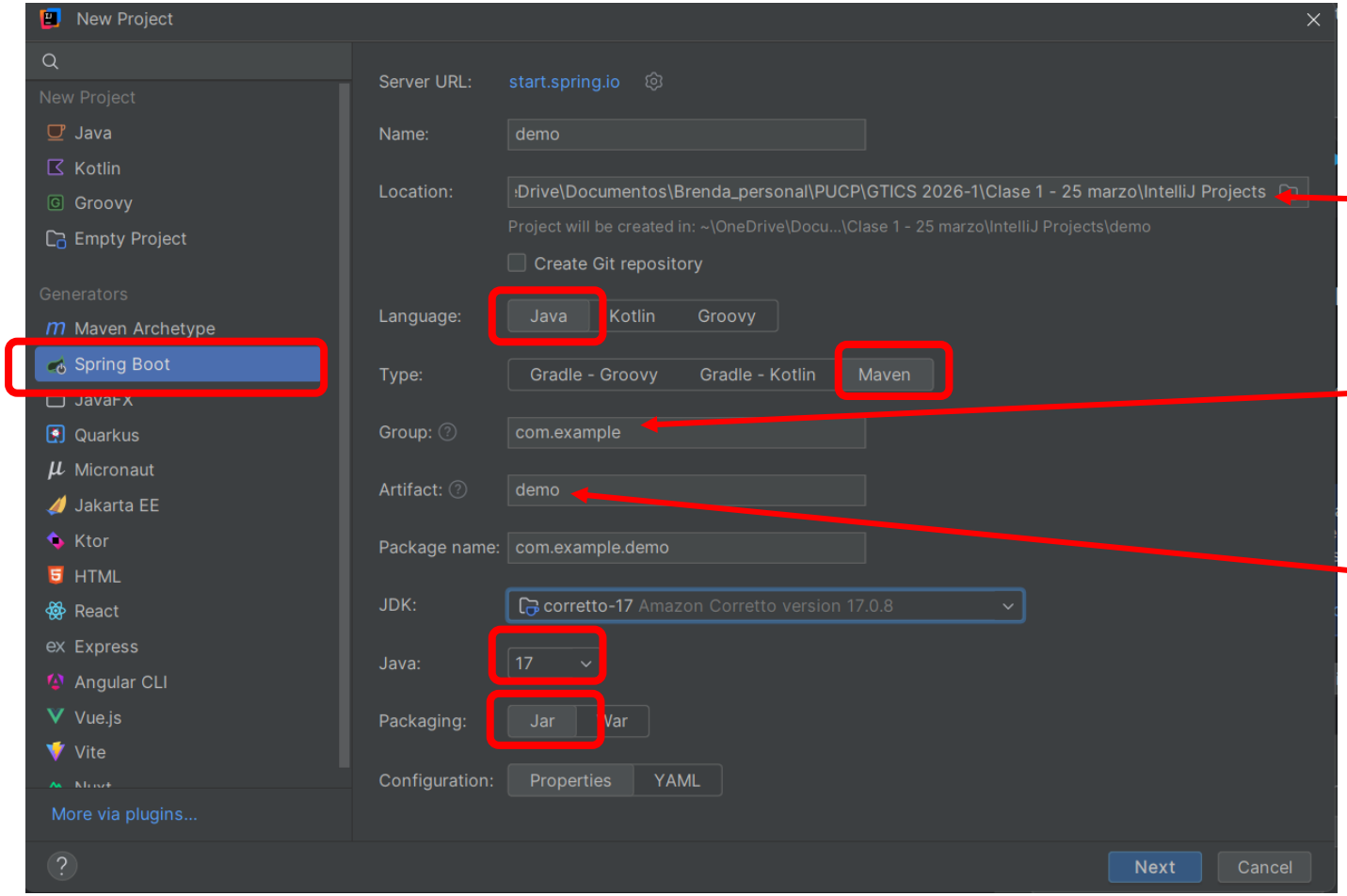
Spring

- Spring Boot es una extensión del framework. Spring elimina las configuraciones (muy extensas en xml) requeridas para configurar una aplicación en Spring.
- Spring Boot incorpora la característica RAD (Rapid Application Development) al framework Spring.
- Algunas de las características de Spring Boot:
  - Dependencias “starter” para simplificar la configuración de la aplicación.
  - Servidor incorporado.
  - Autoconfiguración en la mayoría de los módulos.

Creación de un proyecto con Spring Boot

- Existen varias formas para crear un proyecto con Spring Boot.
  - En el curso se utilizará el wizard de IntelliJ.
  - Se usará la versión **4.x** de Spring Boot.

Creación de un proyecto Spring Boot usando IntelliJ

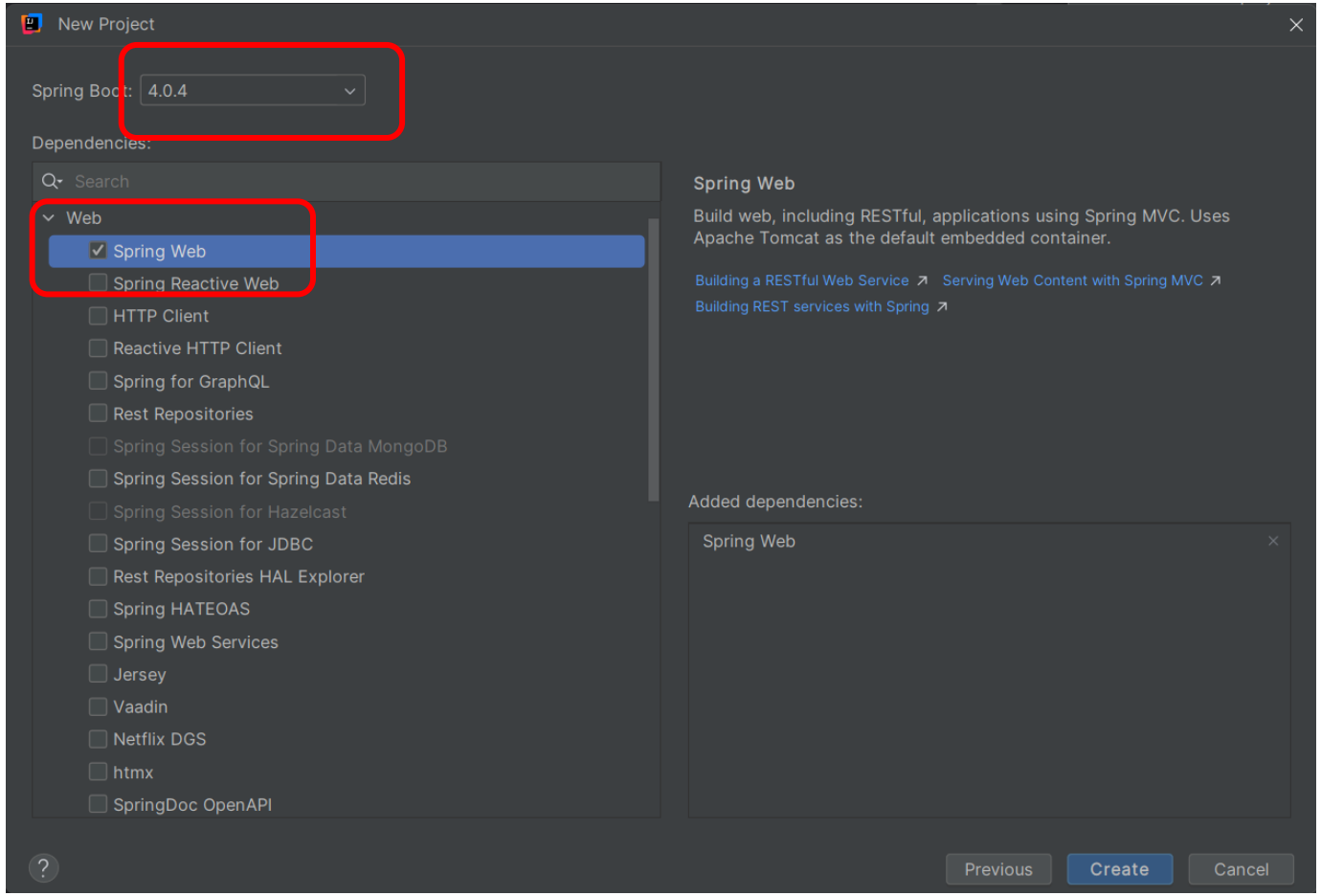


Nombre del proyecto para el IDE

Nombre escrito al inverso del dominio

Nombre del proyecto para maven

Creación de un proyecto Spring Boot usando IntelliJ



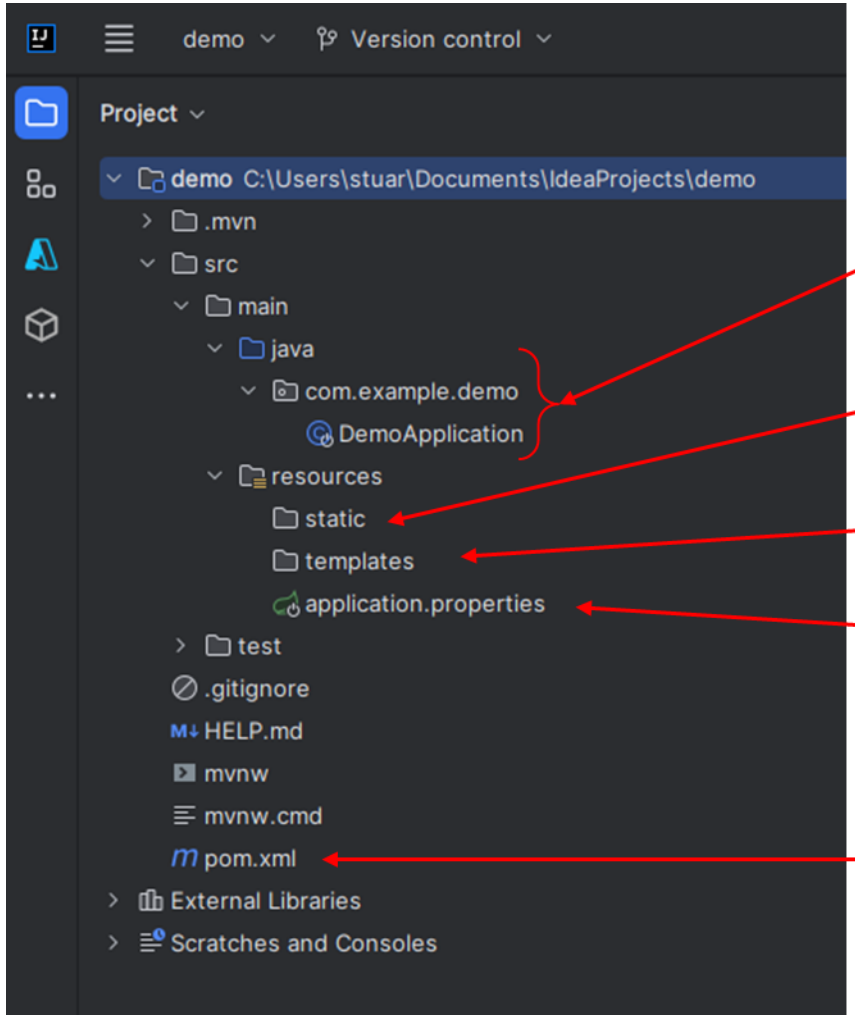
- Activar la dependencia de Spring Web.

Maven

Para los proyectos desarrollados por Spring, se utilizará el gestor de proyectos Maven, el cual permite administrar las librerías (**artefactos**) utilizadas en un proyecto Java, sus dependencias, compilación, empaquetamiento y ejecución.

La descripción del proyecto en Maven se especifica en el archivo **POM.xml**

Estructura de archivos de proyecto



Código de la aplicación (capa controlador y modelo)

Recursos: JS, CSS & otros

Capa Vista: Archivos html

Archivo de configuración de Spring

Archivo de configuración del proyecto

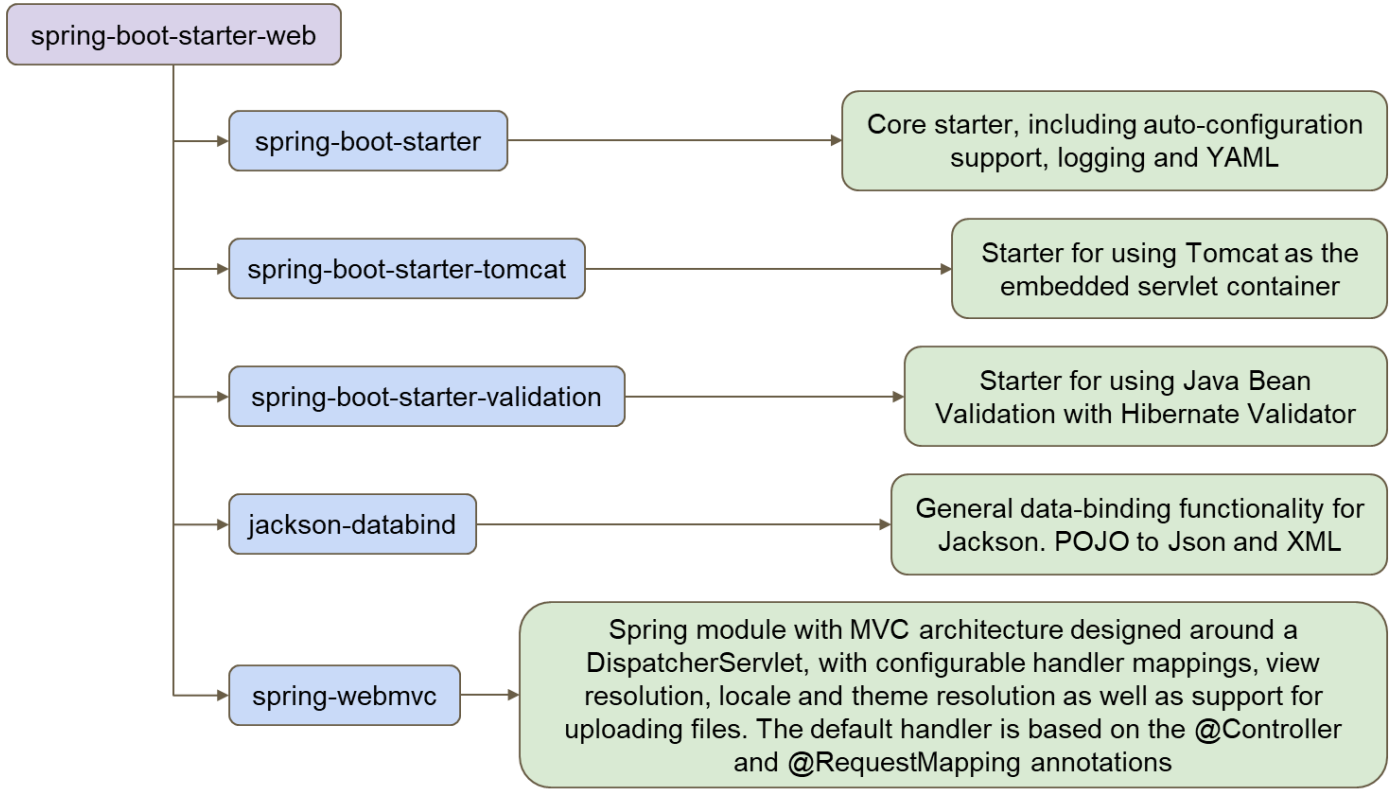
Archivo pom.xml

- El archivo **pom.xml** contiene la configuración de Maven para el proyecto.
- Dentro de las diversas opciones tiene el tag **<dependencies>** donde se gestionan las dependencias necesarias.

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http:
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.0.4</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>
    <groupId>com.example</groupId>
    <artifactId>demo</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>demo</name>
    <description>demo</description>
    <properties>
        <java.version>17</java.version>
    </properties>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>
```

Dependencias: spring-boot-starter-web



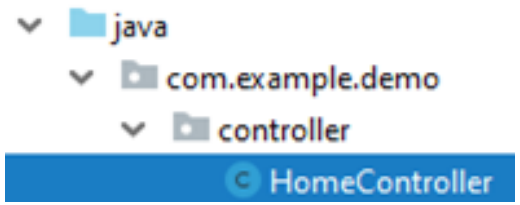
Controladores y Vistas

Controladores

Para empezar a probar la aplicación, se creará un Controlador de nombre **HomeController** en un paquete controller (para gestionar todos los controladores.

Se crea una clase en java (POJO)

```
1  package com.example.demo.controller;
2
3  import org.springframework.stereotype.Controller;
4
5  @Controller
6  public class HomeController {
7
8  }
```



Se le debe colocar la anotación **@Controller**, para indicar que esta clase sea reconocida por Spring como un Controlador, para gestionar las solicitudes HTTP.

Rutas

Para que un controlador pueda gestionar una solicitud HTTP, se debe indicar la ruta (URL web) que identificará al controlador.

Las rutas se pueden mapear a nivel de la clase o de los métodos y se utiliza la anotación **@RequestMapping**

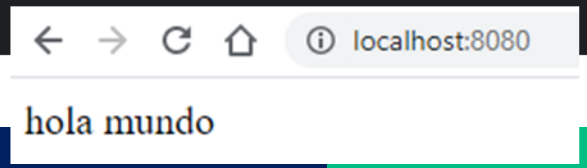
Por ejemplo, para mapear la ruta raíz **"/"**:

La URL para esta ruta será → **ip:puerto/**

```
@Controller
public class HomeController {

    @RequestMapping(value = "/", method = RequestMethod.GET)
    @ResponseBody
    public String paginaPrincipal() {
        return "hola mundo3";
    }
}
```

Si se usa la anotación **@ResponseBody**, lo que se retorna del método, será el cuerpo del mensaje.



@RequestMapping

```
@RequestMapping (value = “RUTA”, method = RequestMethod.MÉTODO)
    public String NOMBREMÉTODO(){

    }
```

Donde:

- **RUTA** es la URL a la cual será mapeado el método o el controlador
- **MÉTODO** es el método http que se desea utilizar (**GET, POST, PUT, DELETE**).

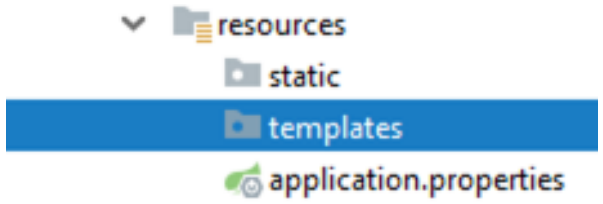
Cada ruta se implementa mediante un método en java donde el nombre solo es referencial.

Vistas

Las vistas serán plantillas html usando “**Thymeleaf**” (similar a JSP usado en software 1).

Las vistas se ubican en la carpeta templates y dentro se puede armar el árbol de carpetas según la necesidad.

Para poder usar las vistas con Thymeleaf, es necesario incluir su dependencia en el **POM.xml**



Observación

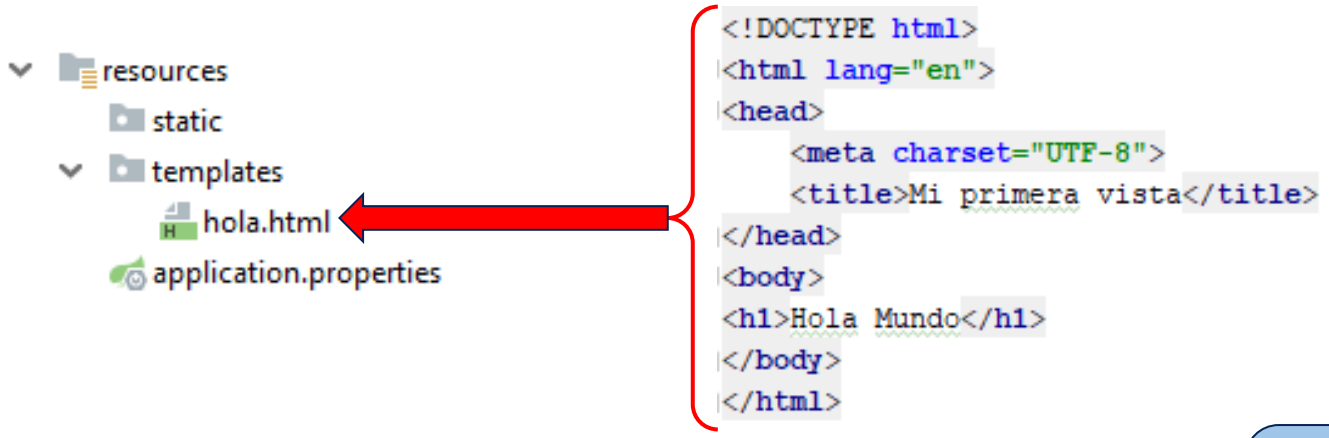
El repositorio de Spring almacena un conjunto de dependencias que ya vienen con autoconfiguración y listas para trabajar integradas con Spring Boot.
Estas las puede encontrar dentro en:

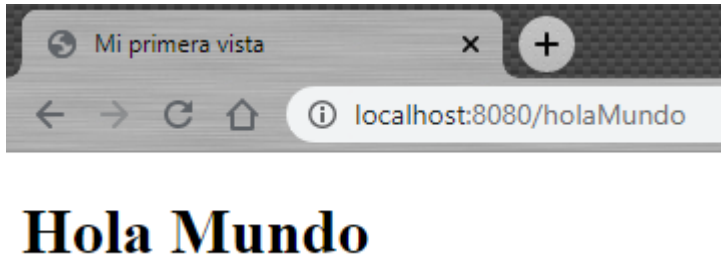
- groupId: **org.springframework.boot**
- artifactId: **spring-boot-starter-XXXXX**

Vistas

Ejemplo:

Mapear la ruta “**/holaMundo**” en el controlador y que aparezca el contenido de la vista **hola.html**



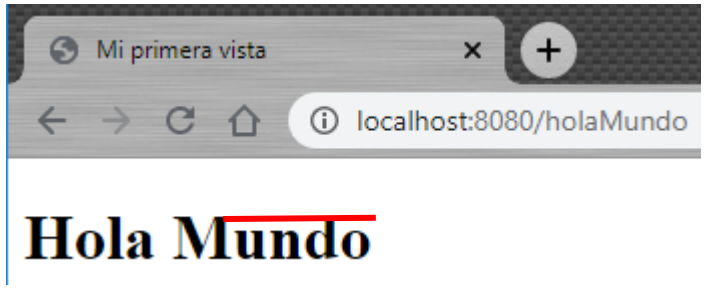


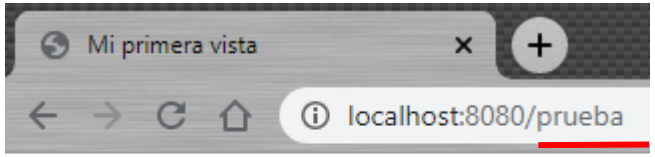
No es necesario indicar la extensión del archivo (**html**), por defecto Spring vincula las vistas de **Thymeleaf** con la extensión **html**.

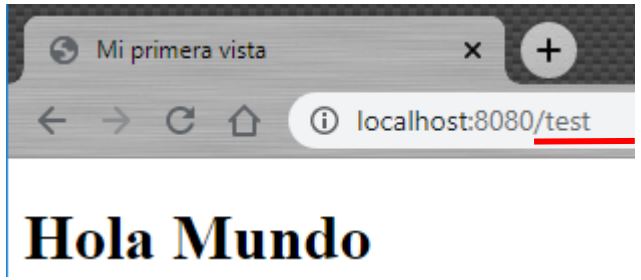
Múltiples rutas a un mismo método

- Spring permite mapear varias rutas a un mismo método usando { }

```
@RequestMapping(value={"/holaMundo","/prueba","test"},method = RequestMethod.GET)
public String holaMundo(){
    return "hola";
}
```







Spring Boot DevTools

Se debe haber dado cuenta que cada vez que realiza un cambio en el código, tiene que estar reiniciando el servidor o refrescando la página web para que se observen los cambios.

Spring posee una herramienta llamada Spring Boot Devtools, la cual permite:

- Deshabilitar el cache en las páginas
- LiveReload para actualizar los cambios en el navegador
- Auto Compilación y despliegue en el servidor.

Para utilizarlo, debe incluirlo en el pom.

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-devtools</artifactId>
    <scope>runtime</scope>
</dependency>
```

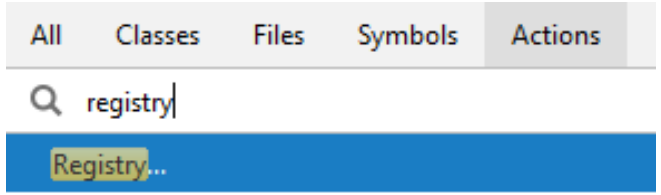
Para información sobre los scopes en maven entre.

Spring Boot Devtools – IntelliJ (ajustes)

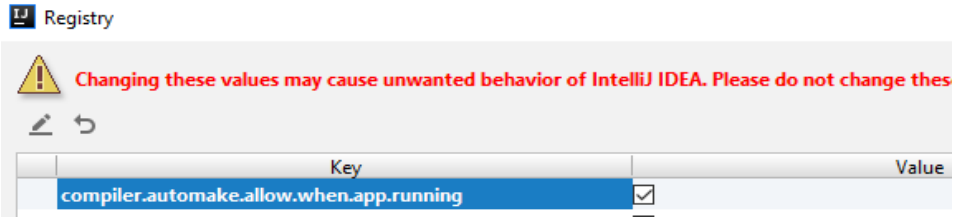
En otros IDEs como eclipse o STS, incluir devtools en el pom sería suficiente; sin embargo, en IntelliJ hay que realizar dos pasos más:

Modificar el registro del IDE:

- Presionar SHIFT + CTRL + A
- En el Pop-up escribir registry y presionar Enter.

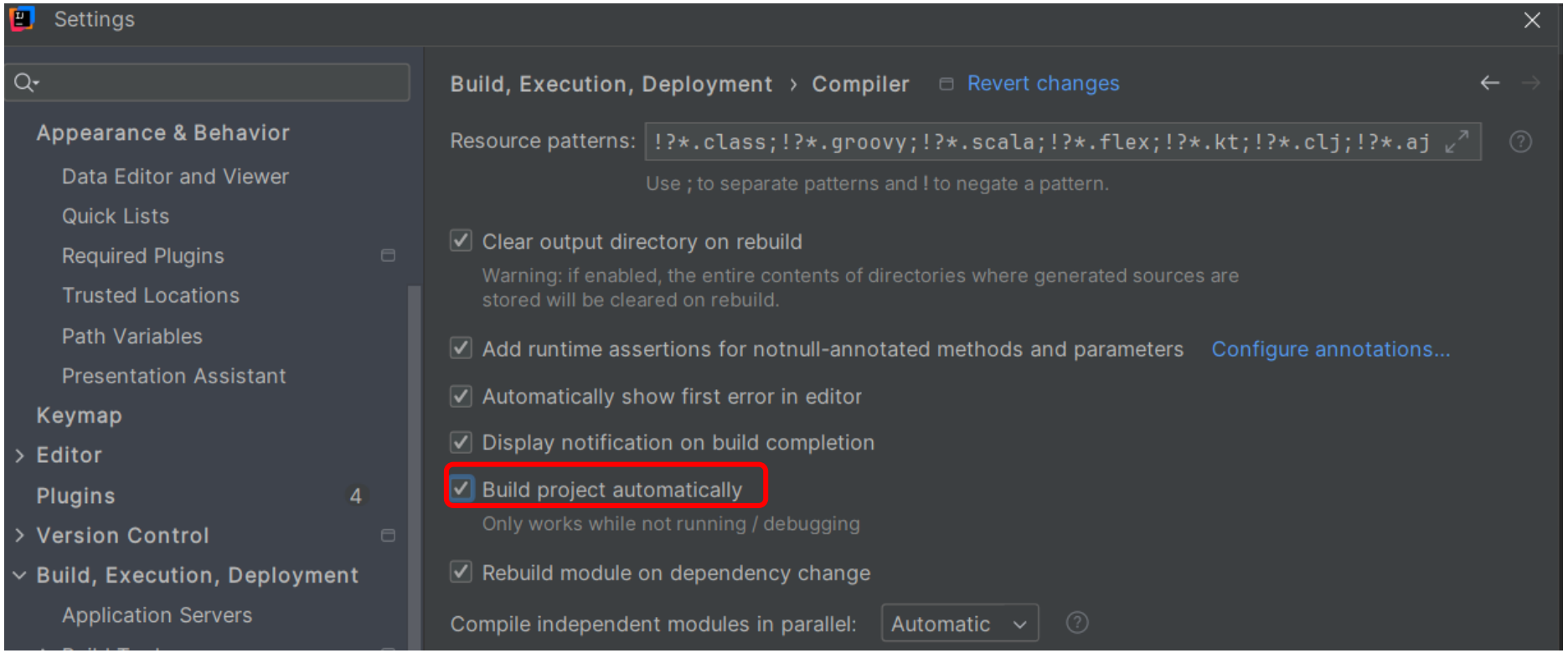


- Buscar: compiler.automake.allow.when.app.running y habilitarlo



Spring Boot Devtools – IntelliJ (ajustes)

2) file → Settings → Build → Compiler → Habilitar: **Build project automatically**



Otros Controladores

- Puede crear cuantos controladores sean necesarios para ordenar su aplicación (no olvide anotarlos con **@Controller**).

```java
@Controller
public class HomeController {

    @RequestMapping(value = "",method = RequestMethod.GET)
    @ResponseBody
    public String paginaPrincipal(){
        return "Hola Mundo";
    }

    @RequestMapping(value={"/holaMundo","/prueba","test"},method = RequestMethod.GET)
    public String holaMundo(){
        return "hola";
    }

}
```

```java
@Controller
public class EmpleadoController {

    @RequestMapping(value = "/empleados", method = RequestMethod.GET)
    public String listaEmpleados(){
        return "empleados/index";
    }
}
```

Ruteo a nivel de clase

- En Spring es posible agrupar un conjunto de rutas a nivel de la clase

```
@Controller
@RequestMapping(value="/empleados", method = RequestMethod.GET)
public class EmpleadoController {

    @RequestMapping(value = "", method = RequestMethod.GET)
    public String listaEmpleados(){
        return "empleados/index";
    }

    @RequestMapping(value="/crearFrm", method = RequestMethod.GET)
    public String formularioCrearEmpleado(){
        return "empleados/crear";
    }

    @RequestMapping(value="/guardar", method = RequestMethod.POST)
    public String guardarEmpleado(){
        return "redirect:/empleados";
    }
}
```

URL para listar empleados: /**empleados**

URL para el formulario: /**empleados/crearFrm**

URL para guardar empleado: /**empleados/guardar** con el método POST (viene de un formulario)

Request Mapping corto

- Se puede reducir la anotación **@RequestMapping**, según el método HTTP utilizado.

```
@RequestMapping(value="/login", method = RequestMethod.GET)
```

```
@GetMapping(value="/login")
```

```
@RequestMapping(value="/login", method = RequestMethod.POST)
```

```
@PostMapping(value="/login")
```

```
@RequestMapping(value="/login", method = RequestMethod.PUT)
```

```
@PutMapping(value="/login")
```

```
@RequestMapping(value="/login", method = RequestMethod.DELETE)
```

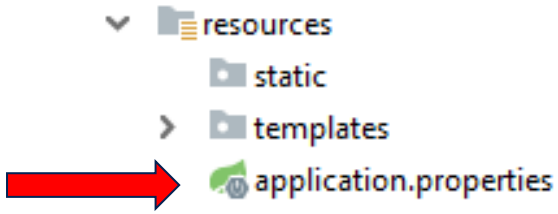
```
@DeleteMapping(value="/login")
```

Application.properties

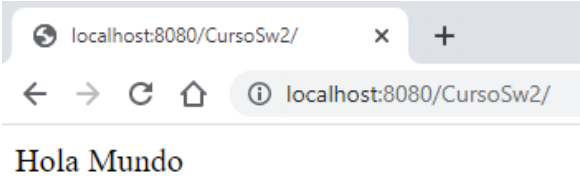
El archivo properties en Spring Boot permite de manera simple, colocar configuraciones al proyecto en formato **yaml**.

Por ejemplo: cambiar el contexto del proyecto a /**CursoSw2**.

En los logs puede ver el resultado:



Y en su navegador web de igual forma:



Recepción de parámetros por GET y POST

Recepción de parámetros por la URL

En Spring existen dos formas de recibir parámetros en la URL:

1. @RequestParam
2. @PathVariable

@RequestParam

```
@GetMapping("/noticia")
public String noticia(@RequestParam("id") int idNoticia,
                      @RequestParam(name = "autor", required = false) String autorNoticia,
                      @RequestParam("mes") String mes){
    System.out.println("id:" + idNoticia + " /autor: " + autorNoticia + " /mes: " + mes);
    return "noticia";
}
```

Las variables esperadas en el URL pueden ser opciones y se registran con **@RequestParam**.

Sin embargo, el valor que le asigna Spring a los parámetros opcionales es "null", por lo tanto, si desea declarar un int como opcional, debe usar su respectiva clase Integer (por ejemplo).

localhost:8080/noticia?id=30&mes=marzo

Console | Endpoints

```
id:30 /autor: null /mes: marzo
```

localhost:8080/noticia?id=30&mes=marzo&autor=Juan

Console | Endpoints

```
id:30 /autor: Juan /mes: marzo
```

localhost:8080/noticia?id=30&autor=Juan

Console | Endpoints

```
olved [org.springframework.web.bind.MissingServletRequestParameterException: Required String parameter 'mes' is not present]
```

@PathVariable

Las variables pueden ser recibidas como parte del URL y **ninguna** puede ser opcional.

```
@GetMapping("/noticia/{id}/{autor}")
public String noticia(@PathVariable("id") int idNoticia,
                      @PathVariable("autor") String autorNoticia){
    System.out.println("id:" + idNoticia + "\nAutor: " + autorNoticia);
return "page";
}
```

En el ejemplo se reciben dos parámetros en la ruta, el ID y el autor. **Ninguno puede ser opcional**.

Spring realizará automáticamente la conversión, del valor ingresado al tipo de dato definido.

Recepción de parámetros por POST

- Para recibir los parámetros enviados por POST, se hace uso de la notación **@RequestParam** (la misma usada para GET)

```html
<h1>Registro</h1>
<form method="POST" action="/persona/guardar">
    <label>Nombre:</label>
    <input type="text" name="nombre" /><br />
    <label>Apellido:</label>
    <input type="text" name="apellido" /><br />
    <label>Dni:</label>
    <input type="text" name="dni" /><br />
    <input type="submit" value="Registrar" />
</form>
```

Registro

Nombre: Carlos
Apellido: Perez
Dni: 80034526
Registrar

```java
@PostMapping("/persona/guardar")
public String guardarPersona(@RequestParam("nombre") String nombrePersona,
                             @RequestParam("apellido") String apellidoPersona,
                             @RequestParam(name = "dni") String dniPersona) {

    System.out.println("Nombre: " + nombrePersona + "\n" +
            "Apellido: " + apellidoPersona + "\n" +
            "Dni: " + dniPersona);

    return "page";
}
```

Console | Endpoints

```
Nombre: Carlos
Apellido: Perez
Dni: 80034526
```

Envío de información del controlador a la vista Objeto Model

Objeto Model

Hasta ahora, hemos recibido información (ya sea por GET o POST) enviada desde la vista (html) a los controladores.

Para enviar información desde los controladores a las vistas, en Spring se hace uso del **Objeto Model**, el cual permite enviar objetos y variables primitivas.

Objeto Model

Primero se debe colocar el objeto Model como atributo en el método de la ruta.

```
@GetMapping("/home")
public String home(Model model) {
```

Y se debe importar de springframework

```
import org.springframework.ui.Model;
```

Luego, para enviar un valor a la vista, se utiliza el método **addAttribute**, el cual recibe dos parámetros: el primero es el nombre con que se enviará la información y el segundo el valor en sí.

Por ejemplo, se enviará el dato “este es un mensaje” en la variable “msg”.

```
@GetMapping("/home")
public String home(Model model) {
    model.addAttribute( s: "msg",  o: "este es un mensaje");
    return "home";
}
```

En la vista

Para mostrar la variable enviada desde el controlador en la vista en Thymeleaf, se hace uso de EL (expression language).

Expression Language es un lenguaje utilizado para interactuar con los datos enviados desde el controlador, mediante el objeto **MODEL**, sin importar el alcance de los atributos (request, session, application).

EL utiliza la expresión **${ }** para mostrar su información.

En la vista

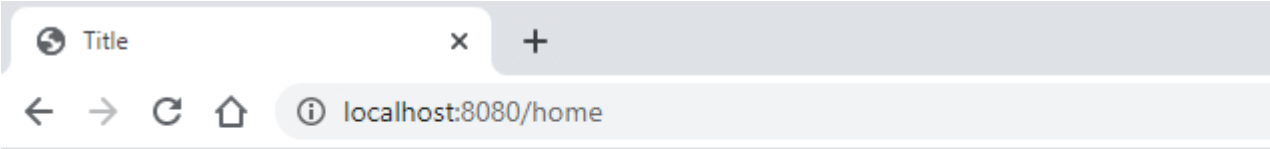
```java
@GetMapping("/home")
public String home(Model model) {
    model.addAttribute( s: "msg",  o: "este es un mensaje");
    return "home";
```

home.html

```html
1  <!DOCTYPE html>
2  <html lang="en" xmlns:th="http://www.thymeleaf.org">
3  <head>
4      <meta charset="UTF-8">
5      <title>Title</title>
6  </head>
7  <body>
8  <h3>Información recibida desde el controlador: <span th:text="${msg}"/></h3>
9  </body>
10 </html>
```

No olvide incluir el namespace

http://www.thymeleaf.org



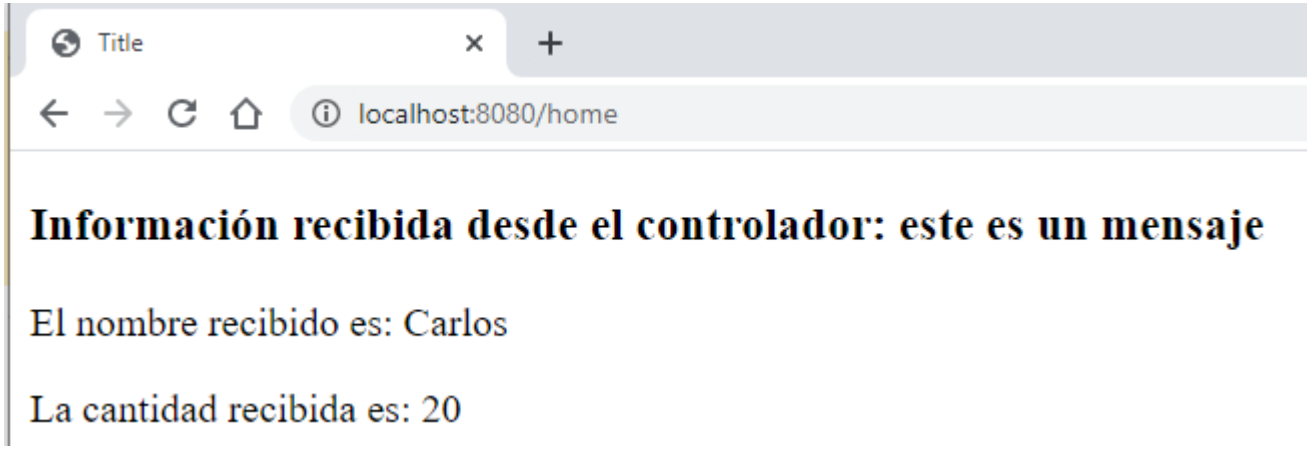
Información recibida desde el controlador: este es un mensaje

Objeto Model

También se pueden enviar varios parámetros:

```
@GetMapping("/home")
public String home(Model model) {
    model.addAttribute( s: "msg",  o: "este es un mensaje");
    model.addAttribute( s: "nombre", o: "Carlos");
    model.addAttribute( s: "cantidad", o: 20);
    return "home";
}
```

```
<h3>Información recibida desde el controlador: <span th:text="${msg}"/></h3>
<p th:text="|El nombre recibido es: ${nombre}|"></p>
<div th:text="'La cantidad recibida es: ' + ${cantidad}"></div>
```



Objeto Model

- Y como se comentó previamente, incluso objetos:

```
@GetMapping("/persona")
public String unaPersona(Model model){
    Persona persona =  new Persona( nombre: "Juan", apellido: "Perez", dni: "12345678", edad: 20);
    model.addAttribute( s: "persona",persona);
    return "home";
}

@GetMapping("/personas")
public String variasPersonas(Model model){
    Persona p1 =  new Persona( nombre: "Juan", apellido: "Perez", dni: "12345678", edad: 20);
    Persona p2 =  new Persona( nombre: "Carlos", apellido: "Mencía", dni: "82564875", edad: 25);
    model.addAttribute( s: "persona",p1);
    model.addAttribute( s: "persona",p2);
    return "home";
}

@GetMapping("/listaPersonas")
public String listaPersonas(Model model){
    ArrayList<Persona> listaPersonas = new ArrayList<>();
    listaPersonas.add(new Persona( nombre: "Juan", apellido: "Perez", dni: "12345678", edad: 20));
    listaPersonas.add(new Persona( nombre: "Carlos", apellido: "Mencía", dni: "82564875", edad: 25));
    listaPersonas.add(new Persona( nombre: "Andrea", apellido: "Huanca", dni: "78649257", edad: 30));
    model.addAttribute( s: "listaPersonas",listaPersonas);
    return "home";
}
```

```
public class Persona {
    private String nombre;
    private String apellido;
    private String dni;
    private int edad;
```

Bibliografía

- Spring
  - https://spring.io/
- Spring Controllers
  - https://docs.spring.io/spring/docs/current/spring-framework-reference/web.html#mvc-controller
  - https://docs.spring.io/spring/docs/3.2.x/spring-framework-reference/html/mvc.html#mvc-controller
- Spring Boot
  - https://docs.spring.io/spring-boot/docs/2.3.0.BUILD-SNAPSHOT/reference/html/getting-started.html#getting-started
- Libro:
  - Siva, P. R. K. (2018). Beginning Spring Boot 2: applications and microservices with the Spring framework. New York: Apress.